

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos

Proyecto

(Micro) sistema de archivos

Integrantes

Campos Cortes Isaac

Martinez Pérez Alejandro

Profesor: Gunnar Eyal Wolf Iszaevich

Fecha de entrega: 21 de mayo de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Entorno de desarrollo y dependencias	2
3.1. Bibliotecas utilizadas	3
3.2. Instalación previa recomendada	3
4. Estructura de FiUnamFS	4
5. Diseño general de la solución	4
5.1. Archivo principal	4
5.2. Módulo <code>superbloque.py</code>	5
5.3. Módulo <code>entrada.py</code>	5
5.4. Módulo <code>disco.py</code>	5
6. Uso de FUSE en el proyecto	5
6.1. Directorio de montaje vacío	6
7. Ejecución del programa	6
7.1. Desmontar el sistema	6
8. Operaciones implementadas	6
8.1. Listar archivos	7
8.2. Leer o extraer un archivo	7
8.3. Agregar un archivo	7
8.4. Eliminar un archivo	7
9. Sincronización y uso de hilos	8
9.1. RLock	8
9.2. Condition	8
10. Pruebas sugeridas	8
11. Limitaciones	9
12. Conclusiones	9

1. Introducción

El proyecto consiste en implementar una herramienta capaz de trabajar con una imagen de disco que contiene el sistema de archivos **FiUnamFS**. Dicha imagen funciona como un pseudodispositivo: no se monta de forma nativa como una memoria USB o una partición convencional, sino que debe ser interpretada por el programa desarrollado.

La solución se implementó en Python utilizando FUSE, lo que permite presentar el contenido de `fiunamfs.img` como un sistema de archivos montado en un directorio del sistema operativo. Con esto, el usuario puede interactuar con los archivos mediante operaciones comunes como listar, copiar, crear, escribir y eliminar.

El enfoque principal del proyecto fue respetar la estructura definida para **FiUnamFS**, separar las responsabilidades del código y emplear mecanismos de sincronización para evitar conflictos cuando las operaciones modifican información compartida.

2. Objetivos

De acuerdo con el planteamiento del proyecto, el programa desarrollado debe cumplir con los siguientes objetivos:

1. Listar los contenidos del directorio del sistema de archivos.
2. Copiar uno de los archivos de dentro de **FiUnamFS** hacia el sistema operativo.
3. Copiar un archivo de la computadora hacia **FiUnamFS**.
4. Eliminar un archivo de **FiUnamFS**.
5. Contar con por lo menos dos hilos de ejecución operando concurrentemente, comunicando su estado mediante mecanismos de sincronización.

En esta implementación, estos objetivos se cubren mediante una interfaz FUSE. Al montar la imagen en un directorio vacío, las operaciones del sistema operativo se traducen a métodos del programa: `readdir` para listar, `read` para extraer información, `create` y `write` para agregar o modificar archivos, y `unlink` para eliminar entradas.

3. Entorno de desarrollo y dependencias

El proyecto fue desarrollado en Python 3 y requiere un entorno Linux con soporte para FUSE. Esto se debe a que FUSE permite que un programa en espacio de usuario publique un sistema de archivos virtual para que el kernel lo exponga como si fuera un sistema de archivos normal.

3.1. Bibliotecas utilizadas

Las bibliotecas empleadas se dividen en dos grupos: bibliotecas estándar de Python y bibliotecas externas necesarias para FUSE.

Biblioteca	Uso dentro del proyecto
<code>os</code>	Manejo de rutas, usuario, grupo y funciones del sistema operativo.
<code>sys</code>	Lectura de argumentos recibidos al ejecutar el programa.
<code>stat</code>	Definición de permisos y tipos de archivo reportados por FUSE.
<code>errno</code>	Devolución de códigos de error compatibles con operaciones de sistema de archivos.
<code>datetime</code>	Conversión de fechas almacenadas en FiUnamFS hacia marcas de tiempo usadas por el sistema.
<code>threading</code>	Implementación de hilos, <code>RLock</code> y <code>Condition</code> para sincronización.
<code>fuse</code>	Biblioteca externa que permite montar el sistema de archivos usando FUSE desde Python.

Además de estas bibliotecas, el proyecto utiliza módulos propios: `disco.py`, `superbloque.py`, `entrada.py` y `herramientas.py`. Estos módulos concentran la lógica específica para interpretar la imagen `fiunamfs.img`.

3.2. Instalación previa recomendada

Antes de ejecutar el proyecto, se debe contar con Python 3 y FUSE instalado. En distribuciones basadas en Debian o Ubuntu, normalmente puede prepararse el entorno con comandos similares a los siguientes:

```
sudo apt update
sudo apt install python3 python3-pip fuse libfuse-dev pkg-config
```

También se requiere el módulo de Python para FUSE. Dependiendo de la distribución, puede instalarse con alguno de estos comandos:

```
sudo apt install python3-fuse
```

o bien:

```
python3 -m pip install fuse-python
```

Si el sistema no permite montar FUSE con el usuario actual, se debe revisar que el módulo `fuse` esté disponible y que el usuario tenga permisos para utilizarlo.

4. Estructura de FiUnamFS

FiUnamFS es un sistema de archivos simple almacenado dentro de una imagen de tamaño fijo. La imagen representa un disco de 1440 KiB y se divide en clusters. Cada cluster mide 2048 bytes, ya que está formado por cuatro sectores de 512 bytes.

La estructura general utilizada por el programa es la siguiente:

Zona	Descripción
Superbloque	Se ubica en el cluster 0. Contiene el nombre del sistema de archivos, versión, etiqueta, tamaño de cluster, número de clusters del directorio y número total de clusters.
Directorio	Se ubica a partir del cluster 1. Contiene las entradas de los archivos almacenados. Cada entrada mide 64 bytes.
Datos	Se encuentra después del directorio. En esta zona se guarda el contenido real de los archivos.

El sistema maneja un directorio plano, por lo que no existen subdirectorios dentro de la imagen. Cada archivo se describe mediante una entrada de directorio que contiene su tipo, nombre, tamaño, cluster inicial y fechas asociadas.

5. Diseño general de la solución

La implementación se organizó en varios módulos para que cada parte del programa tuviera una responsabilidad clara. Esta separación facilita entender el código y reduce el riesgo de mezclar la lógica de FUSE con la interpretación binaria de la imagen.

5.1. Archivo principal

El archivo principal define la clase `FiUnamFs`, que hereda de `Fuse`. Esta clase es la conexión entre el sistema operativo y la imagen `fiunamfs.img`. Cuando el usuario realiza una operación dentro del punto de montaje, FUSE llama al método correspondiente de esta clase.

Las operaciones principales implementadas son:

- `readdir`: lista los archivos disponibles.
- `getattr`: informa si una ruta corresponde al directorio raíz o a un archivo.
- `read`: lee el contenido de un archivo.
- `create`: crea una nueva entrada.

- `write`: escribe datos dentro de un archivo.
- `unlink`: elimina un archivo del sistema.

5.2. Módulo `superbloque.py`

El módulo `superbloque.py` contiene la clase `SuperBloque`. Su función es leer el primer cluster de la imagen y extraer la información general del sistema de archivos.

Durante esta lectura se validan dos datos importantes:

- El nombre del sistema de archivos debe ser `FiUnamFS`.
- El requerimiento pedía que se validara para la versión 26-2 pero que la imagen proporcionada tenía la versión 24-4. Para realizar pruebas de funcionamiento se cambió eso temporalmente pero decidimos dejarlo como 26-2 de acuerdo a la rúbrica.

Esta validación evita que el programa trabaje accidentalmente sobre un archivo que no corresponde al formato esperado.

5.3. Módulo `entrada.py`

El módulo `entrada.py` define la clase `EntradaDir`. Esta clase representa una entrada individual del directorio.

Cada entrada se interpreta a partir de 64 bytes. La clase se encarga de separar esos bytes en campos concretos: tipo de archivo, nombre, tamaño, cluster inicial, fecha de creación y fecha de modificación. También permite crear una nueva entrada, marcar una entrada como eliminada y convertir la información nuevamente a bytes para escribirla en la imagen.

5.4. Módulo `disco.py`

El módulo `disco.py` contiene la clase `Disco`, que concentra las operaciones directas sobre la imagen. Esta clase se encarga de cargar el directorio, buscar archivos por nombre, leer contenido, escribir nuevas entradas, sobrescribir archivos y eliminar entradas.

También administra la sincronización del proyecto. Para esto utiliza un `RLock`, una `Condition`, una lista de operaciones pendientes y un hilo encargado de actualizar el directorio en la imagen cuando ocurre una modificación.

6. Uso de FUSE en el proyecto

FUSE permite que un programa en espacio de usuario implemente un sistema de archivos sin modificar el kernel. En este proyecto, FUSE se utiliza para que la imagen `fiunamfs.img` pueda ser vista desde un directorio del sistema operativo.

El usuario no interactúa directamente con los bytes de la imagen. En su lugar, monta el sistema en un directorio vacío y usa comandos normales como `ls`, `cp`, `cat` o `rm`. FUSE recibe esas operaciones y las redirige al programa en Python.

6.1. Directorio de montaje vacío

Antes de ejecutar el programa se debe crear un directorio que servirá como punto de montaje. Este directorio debe estar vacío. No debe contener archivos importantes, porque mientras el sistema esté montado, lo que se mostrará ahí será el contenido virtual de `FiUnamFS`, no el contenido real del directorio.

Ejemplo:

```
mkdir montaje
```

Para verificar que está vacío:

```
ls -la montaje
```

Debe mostrar únicamente las entradas `.` y `...`. Si contiene archivos, se recomienda crear otro directorio limpio o vaciarlo antes de montar.

7. Ejecución del programa

La forma esperada de ejecución es indicar primero la imagen `fiunamfs.img` y después el directorio vacío donde se montará el sistema.

```
python3 main.py fiunamfs.img montaje
```

7.1. Desmontar el sistema

Al terminar las pruebas, el sistema debe desmontarse correctamente. En Linux se puede usar:

```
umount montaje
```

Es importante desmontar antes de cerrar la terminal o modificar manualmente la imagen, para evitar dejar el punto de montaje en un estado inconsistente.

8. Operaciones implementadas

Una vez montado el sistema, las operaciones se realizan desde el directorio de montaje.

8.1. Listar archivos

Para ver el contenido de `FiUnamFS`:

```
ls montaje
```

Internamente, esta operación llama al método `readdir`, que consulta las entradas válidas cargadas desde el directorio de la imagen.

8.2. Leer o extraer un archivo

Para mostrar el contenido de un archivo de texto:

```
cat montaje/README.org
```

Para copiarlo hacia el sistema operativo:

```
cp montaje/README.org ./README_extraido.org
```

En ambos casos, FUSE llama al método `read`. El programa busca la entrada del archivo, calcula el offset usando su cluster inicial y lee únicamente la cantidad de bytes indicada por el tamaño real del archivo.

8.3. Agregar un archivo

Para copiar un archivo local hacia `FiUnamFS`:

```
echo "archivo de prueba" > prueba.txt  
cp prueba.txt montaje/
```

Esta operación activa `create` y `write`. El programa valida que no exista una entrada con el mismo nombre, busca espacio disponible, escribe los datos en la zona correspondiente y actualiza el directorio.

8.4. Eliminar un archivo

Para eliminar un archivo del sistema montado:

```
rm montaje/prueba.txt
```

Esta operación llama a `unlink`. La entrada se marca como vacía y posteriormente el hilo de escritura sincroniza el directorio con la imagen.

9. Sincronización y uso de hilos

El proyecto incluye concurrencia mediante el módulo `threading`. La clase `Disco` crea un hilo auxiliar llamado `hiloEscritura`, encargado de esperar operaciones pendientes y escribir los cambios del directorio en la imagen.

9.1. RLock

Se utiliza `threading.RLock` para proteger las secciones críticas. Este tipo de bloqueo permite que un mismo hilo adquiera el candado más de una vez si una función protegida llama a otra función que también requiere el mismo lock.

En el proyecto, el `RLock` protege operaciones que modifican entradas, escriben datos o actualizan el directorio.

9.2. Condition

La variable `Condition` permite que el hilo de escritura espere hasta que exista una operación pendiente. Cuando se agrega, modifica o elimina un archivo, el programa agrega una operación `sync` a la lista de pendientes y notifica al hilo.

El flujo general es el siguiente:

1. Una operación modifica una entrada del directorio.
2. Se agrega `sync` a la lista de operaciones pendientes.
3. Se notifica a la condición.
4. El hilo de escritura despierta.
5. El hilo llama a `actualizarDisco` para guardar el directorio actualizado en la imagen.

Con esto, el programa evita que varias operaciones escriban simultáneamente sobre la misma estructura del sistema de archivos.

10. Pruebas sugeridas

Las siguientes pruebas permiten verificar el comportamiento principal del proyecto. Se recomienda hacerlas con una copia de la imagen original.

```
cp fiunamfs.img fiunamfs_prueba.img
mkdir montaje
python3 main.py fiunamfs_prueba.img montaje
```

En otra terminal:

```
ls montaje
cat montaje/README.org
cp montaje/README.org ./README_extraido.org

echo "hola desde FUSE" > prueba.txt
cp prueba.txt montaje/
ls montaje
cat montaje/prueba.txt
rm montaje/prueba.txt
ls montaje
```

Al finalizar:

```
umount montaje
```

Estas pruebas cubren las operaciones solicitadas: listar, copiar desde FiUnamFS, copiar hacia FiUnamFS y eliminar.

11. Limitaciones

La implementación cumple con el objetivo del proyecto, pero mantiene algunas limitaciones propias de FiUnamFS y del alcance desarrollado:

- El sistema maneja un directorio plano; no se implementan subdirectorios.
- Los nombres de archivo están limitados a 15 caracteres.
- El formato espera cadenas ASCII, por lo que no se recomienda usar acentos ni caracteres Unicode en nombres de archivo.
- La asignación de datos se maneja de forma contigua.
- El programa depende de FUSE, por lo que está orientado principalmente a entornos Linux.
- La imagen debe corresponder al formato esperado por el programa; de lo contrario, se rechaza durante la lectura del superbloque.

12. Conclusiones

Este proyecto permitió implementar de forma práctica varios conceptos de sistemas operativos. Por un lado, se trabajó con la estructura interna de un sistema de archivos: superbloque,

directorio, entradas, clusters y zona de datos. Por otro lado, se integró FUSE para exponer la imagen como un sistema de archivos utilizable desde comandos normales del sistema operativo.

La parte de sincronización fue importante porque las operaciones de escritura no deben modificar el directorio de manera desordenada. El uso de `RLock`, `Condition` y un hilo dedicado a sincronizar cambios permitió separar la operación solicitada por FUSE de la actualización persistente del directorio dentro de la imagen.

En conjunto, la implementación muestra cómo operaciones aparentemente simples, como copiar o eliminar un archivo, requieren validar estructuras internas, calcular posiciones dentro del disco, actualizar metadatos y proteger el acceso concurrente a los datos compartidos.